VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Chosen Software: VLC Media Player

Student Name: Mukul
Registration Number: 24BCE10027

Course: Open Source Software

Unit Coverage: 1 – 5

Date of Submission: March 29, 2026

Chapter 1: Introduction and Overview

I have been using VLC since school to watch movies and videos. My laptop has never been able to play some weird video formats, but VLC always handles them. I used to just install it and forget about it. For this project I actually looked into how it was made, who made it, and why they decided to make it free for everyone.

Open source software is something I always heard about but never fully understood. Now after reading about it I feel like it is actually a really important idea. When code is open anyone can look at it, fix bugs in it, and make it better. This is very different from something like Windows Media Player where you just have to trust Microsoft that it is working correctly.

1.1 Chosen Software: VLC Media Player

VLC is a free media player that was made by a group of students at a school in France called Ecole Centrale Paris. The project started in 1996 and the first public version came out in 2001. Right now it has been downloaded more than 5 billion times which is honestly a crazy number for something that was made by students.

What makes VLC different from other players is that it can play almost anything. MP4, MKV, AVI, FLV, even damaged or partially downloaded files. You don't need to install extra codecs. Everything is built in. And it runs on Windows, Linux, Mac, Android and iPhone too.

Chapter 2: Research Analysis

2.1 Part A – Origin and Philosophy

2.1.1 The Problem Before VLC

Before VLC existed, playing videos on a computer was really annoying. You needed different codec packs for different video types. Sometimes you would install a codec and it would break something else. Windows Media Player was the default option but it couldn't play most of the formats people actually used. There was no single player that worked reliably for everything.

The students who made VLC were working on a research project about video streaming over a network. They needed a player that could handle video streams and also different file types. Since nothing available at the time did what they needed, they just built their own.

2.1.2 The Origin Story of VLC

The project started inside an organization called VideoLAN. It was originally built as a student project. When the school project ended the students decided to release it publicly under an open source license instead of just letting it die. That decision in 2001 changed everything because suddenly anyone anywhere could use it.

The main goals they had when building it were:

- It should play any video format without needing extra software
- It should work on all major operating systems

- It should be able to stream video over a network
- It should stay completely free

2.1.3 License Analysis: GPL v2

VLC uses the GNU General Public License version 2 which is the same license that Linux uses. GPL is what people call a copyleft license. This means if you take VLC's code and change it and then share your version, you are required to share the source code of your changes too. You can't just take the free code, improve it, and then sell a closed version.

The four freedoms that define truly free software are the freedom to run the program, the freedom to study its source code, the freedom to share it with others, and the freedom to share your modifications. VLC satisfies all four of these.

The GPL is different from a license like MIT. MIT lets you do basically anything including taking the code and making it proprietary. GPL prevents that. Since VLC is something everyone relies on, GPL makes sure no company can take it and lock it down.

There is a famous phrase in open source: free as in beer versus free as in speech. A free app on your phone might cost zero rupees but the company controls everything about it. VLC is free as in speech meaning you have real rights over it.

2.1.4 The Ethics of Open Source

I think tools that millions of people depend on every day should be open source. Media players, web browsers, operating systems,

these are things that form the base of how people use computers. If one company controls all of that they have too much power.

At the same time I understand that developers need to eat. I think it is fine for companies to build paid products on top of open source as long as they contribute back. The VLC model works because the core stays free and community maintained while some companies offer paid support or integrations.

We often talk about standing on the shoulders of giants in engineering. Every time I write code I am using tools that other people spent years building and then gave away for free. I feel like that comes with some responsibility to give back in whatever way I can even if it is just bug reports or documentation.

2.2 Part B – Linux Footprint

2.2.1 Installation Method

Installing VLC on Ubuntu is very simple. You just open the terminal and run:

```
sudo apt update
sudo apt install vlc
```

It is in the default Ubuntu repositories so you don't need to add any extra sources. You can also install it using snap:

```
sudo snap install vlc
```

2.2.2 Directory Structure

After installing VLC the main program lives at /usr/bin/vlc. The configuration for each user is stored in a hidden folder inside the home directory at ~/.config/vlc/. This separation of system files and user settings is important because it means one user's settings don't affect other users on the same machine.

Plugin files and libraries are kept at /usr/lib/x86_64-linux-gnu/vlc/ and shared resources like icons and data files are at /usr/share/vlc/.

2.2.3 User and Permissions

VLC runs under whoever is logged in at the time. It doesn't need root or admin access to play a video. This is the right way to design software. If a normal user runs VLC and something goes wrong or gets exploited, it can only touch files that user has access to. It can't mess up the whole system.

2.2.4 Service Behavior

VLC is not a background service. It doesn't run all the time eating RAM and CPU. You open it when you need it and close it when you are done. This is how most good desktop tools work. The only exception is if you explicitly use it as a streaming server, but that is a separate thing.

2.2.5 Update Model

Security updates and bug fixes for VLC come through the normal apt package manager. When you run sudo apt upgrade it gets updated along with everything else. You never have to manually download anything or run a sketchy installer file from the internet.

2.3 Part C – The FOSS Ecosystem

2.3.1 Dependencies

VLC depends on a lot of open source libraries to do its job. The most important ones are:

- libavcodec and libavformat from FFmpeg for decoding different video and audio formats
- libx264 and libvpx for encoding video
- Qt5 for the graphical interface on desktop
- OpenSSL for handling secure connections when streaming
- zlib for file compression

All of these are themselves open source projects. VLC is basically a collection of open source tools assembled together with some extra code on top. This is how most FOSS software works, building on top of what others already made.

2.3.2 What VLC Enables

VLC has directly influenced how media is handled on the internet. The FFmpeg project which VLC depends on is also used by YouTube, Facebook, and basically every major platform for video processing. A lot of the codec support in browsers today comes from work that started in projects like VLC and FFmpeg.

VLC also supports RTSP, HTTP, and HLS streaming which are the same protocols used for live video streaming. You can actually use VLC to set up a simple streaming server on your local network. This stuff is relevant when you are building anything with live video.

2.3.3 Community and Governance

VLC is maintained by a non-profit organization called VideoLAN which is registered in France. Development happens on a mailing list and through their bug tracker at trac.videolan.org. Anyone can submit a patch. The core team reviews it and decides whether to merge it.

The project is mostly volunteer driven but some developers work on it professionally for companies that depend on VLC. This kind of mixed model is very common in open source.

2.4 Part D – Open Source vs Proprietary

I compared VLC with Windows Media Player since that is the most common alternative most people have used.

Feature	VLC (Open Source GPL v2)	Windows Media Player (Proprietary)
Cost	Completely free, no limits	Comes with Windows but tied to Microsoft
Format Support	Plays almost every format without extra installs	Limited, needs codec packs for many formats
Platform	Windows, Linux, Mac, Android, iOS	Windows only
Security	Code reviewed by community, fast bug fixes	Only Microsoft can check the code
Customization	Full source available, you can change anything	No source access, no modification allowed
Updates	Regular, through system package manager	Comes with Windows updates, slow
Streaming	Full network streaming support built in	Very limited streaming options

Honestly VLC wins in almost every category. The only time I would use Windows Media Player is if someone forced me to. Even Microsoft eventually gave up trying to make it a proper player and it barely gets updated anymore. VLC still gets regular releases.

Chapter 3: Practical Shell Scripts

For the practical part of this project I wrote five shell scripts. These helped me understand how to actually work in the Linux terminal and also how to interact with installed software using commands. I am going to show each script and explain what it does.

3.1 Script 1: System Identity Report

This script collects basic information about the system and prints it out neatly. It uses variables to store the results of commands and then displays everything in a formatted way.

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Mukul | Course: Open Source Software

STUDENT_NAME="Arjun Mehta"
SOFTWARE_CHOICE="VLC Media Player"

KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DISTRO=$(uname -o)
HOME_DIR=$HOME
CURRENT_DATE=$(date)

echo "================================"
echo " Open Source Audit - $STUDENT_NAME"
echo "================================"
echo "Software Choice : $SOFTWARE_CHOICE"
echo "Distribution   : $DISTRO"
echo "Kernel         : $KERNEL"
echo "Current User   : $USER_NAME"
echo "Home Directory : $HOME_DIR"
echo "Uptime         : $UPTIME"
echo "Date and Time  : $CURRENT_DATE"
echo "--------------------------------"
echo "VLC is distributed under the GPL v2 license."
echo "================================"
```

```
read -p "Press Enter to exit..."
```

This script shows how to use variables and command substitution in bash. The $(command) syntax is what lets you run a command and store its output in a variable. Without that you would have to manually type all the system info yourself.

3.2 Script 2: Package Inspector

This script checks whether VLC is installed on the system and shows details about the package. It uses if-else and a case statement.

```
#!/bin/bash
# Script 2: Package Inspector

PACKAGE="vlc"

if dpkg -l "$PACKAGE" &>/dev/null || command -v "$PACKAGE" &>/dev/null; then
    echo "$PACKAGE is installed."
    if command -v dpkg &>/dev/null && dpkg -s "$PACKAGE" &>/dev/null; then
        dpkg -s "$PACKAGE" | grep -E '^Version|^Description' | head -n 2
    else
        echo "Version: $($PACKAGE --version 2>&1 | head -1)"
    fi
else
    echo "$PACKAGE is NOT installed."
fi

case $PACKAGE in
    vlc) echo "VLC: media player that can handle basically any format you throw at it"
;;
    git) echo "Git: the version control system that holds everything together" ;;
    firefox) echo "Firefox: still fighting to keep the web open and free" ;;
    *) echo "$PACKAGE: part of the open source ecosystem" ;;
esac

read -p "Press Enter to exit..."
```

The &>/dev/null part just hides the output of those check commands. Without it you would see a bunch of text every time the script runs. The case statement is basically a cleaner way to write a bunch of if-else checks on the same variable.

3.3 Script 3: Disk and Permission Auditor

This one goes through a list of directories using a for loop and prints out the permissions and disk usage for each one. It also checks if the VLC config folder exists.

```bash
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
echo "Directory Audit Report"
echo "======================="

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        FS_USAGE=$(df -h "$DIR" 2>/dev/null | awk 'NR==2 {print $5}')
        echo "$DIR | Perms: $PERMS | Size: $SIZE | Used: $FS_USAGE"
    else
        echo "$DIR does not exist on this system"
    fi
done

VLC_CONFIG="$HOME/.config/vlc"
if [ -d "$VLC_CONFIG" ]; then
    PERMS=$(ls -ld "$VLC_CONFIG" | awk '{print $1, $3, $4}')
    echo "VLC config exists. Permissions: $PERMS"
else
    echo "VLC config not found at $VLC_CONFIG"
fi
```

```
read -p "Press Enter to exit..."
```

The awk command is really useful for grabbing specific columns from text output. ls -ld gives you a long line of info about a directory and awk lets you pick just the parts you want by column number. The du and df commands give you storage info.

3.4 Script 4: Log File Analyzer

This script takes a file path and a keyword as arguments from the command line and counts how many times that keyword shows up in the file. It reads line by line using a while loop.

```
#!/bin/bash
# Script 4: Log File Analyzer

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    read -p "Press Enter to exit..."
    exit 1
fi

RETRY=0
while true; do
    if [ -s "$LOGFILE" ]; then break; fi
    echo "File is empty, retrying in 2 seconds..."
    sleep 2
    RETRY=$((RETRY + 1))
    if [ $RETRY -ge 3 ]; then
        echo "File still empty after 3 tries. Exiting."
        exit 1
```

```
    fi
done

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Found '$KEYWORD' $COUNT times in $LOGFILE"

if [ $COUNT -gt 0 ]; then
    echo "--- Last 5 matches ---"
    grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
fi

read -p "Press Enter to exit..."
```

The ${2:-"error"} part is how you set a default value for a variable. If the user doesn't give a second argument the script uses error as the keyword. The IFS= read -r LINE pattern is the proper way to read a file line by line without losing spaces or special characters.

3.5 Script 5: Manifesto Generator

The last script asks the user three questions and then puts their answers together into a short statement and saves it to a text file.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator

echo "Answer three questions to generate your manifesto."
read -p "1. One open source tool you use daily: " TOOL
read -p "2. One word for what freedom means to you: " FREEDOM
read -p "3. One thing you would build and share freely: " BUILD
```

```
DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "### My Open Source Manifesto ###" > "$OUTPUT"
echo "Date: $DATE" >> "$OUTPUT"
echo "I use $TOOL every day and I never paid for it." >> "$OUTPUT"
echo "Freedom to me means $FREEDOM and open source is built on that." >> "$OUTPUT"
echo "If I could build anything, I would make $BUILD and give it away." >> "$OUTPUT"

echo "Saved to $OUTPUT"
cat "$OUTPUT"
read -p "Press Enter to exit..."
```

The > redirect creates a new file and the >> appends to it. If you use > for every line you would overwrite the file each time and only have the last line. The read -p command is what makes the script interactive by asking the user for input while it is running.

Chapter 4: Conclusion

Before this project I just thought of VLC as the app you install when Windows Media Player can't open a file. Now I see it differently. It is a project that started as a student experiment in France and became something that billions of people use without paying a single rupee for it.

The GPL license is what made that possible. Because of GPL nobody can take VLC and make it proprietary. The community can always build on it, fix it, and keep it free. That is a pretty powerful idea.

The shell scripting part was actually the most useful thing for me personally. I had only ever typed individual commands before. Writing a script that does multiple things automatically, handles errors, and reads files line by line showed me how much you can do with basic bash. Tools like awk and grep that look intimidating at first are actually pretty logical once you see a few examples.

Overall this project changed how I think about the software I use every day. Most of it is built on open source. The internet basically runs on it. It feels good to understand that now instead of just taking it for granted.